

D793

Formal Languages Overview

Armored Drill

8 concepts · 12 items · 4 transfer items

What this is. Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

Source. Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

Target word _____	The one fact that decides _____	Answer _____	[] sure [] unsure [] guess
-----------------------------	---	------------------------	----------------------------------

Items

1.

A trading firm needs a service with the fastest possible, predictable execution, shipped as a standalone binary. The team's style guide uses four-space indentation. Which type of language fits best?

- A. Assembly for everything, since lower level is always better.
- B. An interpreted scripting language.
- C. A compiled language that produces native machine code.
- D. Whichever language matches the indentation style.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

2.

A 25-line Scala function computes a discount from its inputs, but it also writes to a global log and increments a global counter. In functional programming terms, what is the issue?

- A. It has side effects, so it is not a pure function.
- B. Nothing; it returns a value, so it is pure.
- C. It is too long to be functional.
- D. It should use recursion instead of loops.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

3. [TRANSFER — reversed stem / different variable]

Which of these functions is pure?

- A. `sortInPlace(list)`, which reorders the list passed in and returns it.
- B. `loadPrices()`, which reads a file.
- C. `now()`, which returns the current time.
- D. `square(x)`, which returns $x \times x$.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

4.

A Python service crashed in production when a function received text instead of a number. The bug sat in code that ran only on the last day of each month. The team wants errors like this caught before the program runs. Which language feature helps?

- A. Static type checking.
- B. Run the monthly code on a different day.
- C. More detailed logging.
- D. Dynamic typing.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

5. [TRANSFER — reversed stem / different variable]

In some language, a type error in a rarely used branch is reported only when that branch actually runs. What kind of type checking is this?

- A. No typing at all; the language has no types.
- B. Dynamic type checking.
- C. Static type checking.
- D. Weak typing.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

6.

An analyst writes a SQL query asking for the names of all customers in Ohio. The table holds 3 million rows on fast storage. What best describes how SQL handles this?

- A. It is compiled into a standalone program the analyst runs.
- B. It is procedural: the analyst controls the order rows are scanned.
- C. Its behavior depends mainly on the storage speed.
- D. It is declarative: the analyst states the result, and the database chooses how to find it.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

7.

A payroll system for 1,200 employees has Hourly and Salaried subclasses of Employee, each with its own calculatePay(). Code loops over a list of Employee objects and calls calculatePay() on each without checking its type. Which principle is this?

- A. Scalability, because there are 1,200 employees.
- B. Polymorphism.
- C. Encapsulation.
- D. Inheritance alone.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

8.

A team ran 10,000 automated tests on a routine that will help control medication dosing, and all passed. The tests ran on a new build server. What can the team correctly claim?

- A. It is proven; verification only applies to hardware.
- B. The routine is correct for all inputs, since 10,000 is a lot.
- C. No failures were found on those inputs; formal verification would be needed to prove correctness.
- D. The new build server makes the results a proof.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

9. [TRANSFER — reversed stem / different variable]

What does formal verification provide that testing does not?

- A. Bugs found by running sample inputs.
- B. Proof that the program meets its specification for all inputs.
- C. Proof that the specification is what users actually want.
- D. Faster program execution.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

10.

A loop has been proven so that if it finishes, its result satisfies the postcondition. No one has proven that it always finishes. The proof was reviewed by three engineers. What has been shown?

- A. Nothing; without termination the proof is worthless.
- B. Total correctness.
- C. Total correctness, because three engineers reviewed it.
- D. Partial correctness.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

11. [TRANSFER — reversed stem / different variable]

What must be added to a partial-correctness proof to make it a total-correctness proof?

- A. A proof that the program always terminates, such as a bounded value that strictly decreases each loop.
- B. A larger test suite.
- C. A stronger loop invariant.
- D. A weaker postcondition.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

12.

A team must check that input strings have balanced parentheses nested to any depth. A junior developer proposes a regular expression. The input comes from a web form built last year. Which statement is correct?

- A. A careful regular expression can match any depth.
- B. Rebuild the web form first.
- C. A regular expression cannot do this; it needs a context-free grammar or a stack-based parser.
- D. It needs a full context-sensitive grammar.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

STOP

Answers and dissection follow. Finish every item and every strip first.

Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

1. Answer C Concept 1

A trading firm needs a service with the fastest possible, predictable execution, shipped as a standalone binary. ~~The team's style guide uses four space indentation.~~ Which type of language fits best?

Target word: language type for the requirement **Decides:** Speed and a standalone binary.

A	Absolute language	Costly to build and maintain; compiled high-level code meets the need.
B	Confusion set	Needs an interpreter at run time and is usually slower.
C	Correct	Ahead-of-time compilation gives speed and a standalone binary.
D	Camouflage	Formatting rules do not decide the language.

2. Answer A Concept 2

~~A 25-line Scala~~ function computes a discount from its inputs, but it also writes to a global log and increments a global counter. In functional programming terms, what is the issue?

Target word: why it is not functional style **Decides:** It changes state outside itself.

A	Correct	Writing to globals breaks purity.
B	Dropped element	Ignores the side effects.
C	Camouflage	Length does not decide purity.
D	Confusion set	A common functional habit, but not the problem here.

3. Answer D Concept 2 · transfer

Which of these functions is pure?

Target word: identify a pure function **Decides:** Same input, same output, no side effects.

A	Inverted operation	Mutates its input instead of returning new data.
B	Adjacent category	Depends on the outside world.
C	Confusion set	Same call gives different results; it reads hidden state.
D	Correct	Depends only on its input and changes nothing else.

4. Answer A Concept 3

A Python service crashed in production when a function received text instead of a number. ~~The bug sat in code that ran only on the last day of each month.~~ The team wants errors like this caught before the program runs. Which language feature helps?

Target word: catch type errors before run time **Decides:** The goal is detection before execution.

A	Correct	Types are checked before the code runs, whatever the branch.
B	Camouflage	The date does not affect type safety.
C	Adjacent category	Records the crash after it happens.
D	Inverted operation	That is what let the bug wait until run time.

5. Answer B Concept 3 · transfer

In some language, a type error in a rarely used branch is reported only when that branch actually runs. What kind of type checking is this?

Target word: name the typing discipline **Decides:** Errors appear at run time.

A	Absolute language	It clearly has types; it checks them late.
B	Correct	Types are checked as code executes.
C	Inverted operation	Static checks would flag it before running.
D	Confusion set	Weak typing is about implicit conversions, not timing.

6. Answer D Concept 4

An analyst writes a SQL query asking for the names of all customers in Ohio. ~~The table holds 3 million rows on fast storage.~~ What best describes how SQL handles this?

Target word: nature of a query language **Decides:** The query states the result, not the steps.

A	Adjacent category	Queries run inside the database engine.
B	Inverted operation	SQL does not specify loops or scan order.
C	Camouflage	Storage affects speed, not the language's nature.
D	Correct	The query optimizer picks the execution plan.

7. Answer B Concept 5

A payroll system ~~for 1,200 employees~~ has Hourly and Salaried subclasses of Employee, each with its own calculatePay(). Code loops over a list of Employee objects and calls calculatePay() on each without checking its type. Which principle is this?

Target word: name the OOP principle **Decides:** One call runs each subtype's own version.

A	Camouflage	Headcount is not a language principle.
B	Correct	The same call dispatches to each subclass's method.
C	Confusion set	Encapsulation hides internal state; it is not about dispatch.
D	Adjacent category	Inheritance creates the subclasses; the type-free call is polymorphism.

8. Answer C Concept 6

A team ran 10,000 automated tests on a routine that will help control medication dosing, and all passed. ~~The tests ran on a new build server.~~ What can the team correctly claim?

Target word: what passing tests show **Decides:** Tests cover samples, not all inputs.

A	Confusion set	Formal verification applies to software too.
B	Absolute language	No finite test set covers all inputs.
C	Correct	Testing shows bugs are present, never that they are absent.
D	Camouflage	The server does not change what tests can show.

9. Answer B Concept 6 · transfer

What does formal verification provide that testing does not?

Target word: benefit of verification **Decides:** A proof over all inputs against the spec.

A	Inverted operation	That describes testing.
B	Correct	Verification reasons about every case, not samples.
C	Confusion set	That is validation; verification assumes the spec.
D	Adjacent category	Verification is about correctness, not speed.

10. Answer D Concept 7

A loop has been proven so that if it finishes, its result satisfies the postcondition. No one has proven that it always finishes. ~~The proof was reviewed by three engineers.~~ What has been shown?

Target word: kind of correctness **Decides:** No termination proof.

A	Absolute language	Partial correctness is still a real result.
B	Confusion set	Total correctness also requires proving termination.
C	Camouflage	Reviewers cannot supply the missing termination proof.
D	Correct	Correct if it terminates, with termination unproven.

11. Answer A Concept 7 · transfer

What must be added to a partial-correctness proof to make it a total-correctness proof?

Target word: missing piece **Decides:** Termination.

A	Correct	Termination plus partial correctness equals total correctness.
B	Confusion set	Tests cannot prove termination for all inputs.
C	Adjacent category	Invariants support the result, not termination.
D	Inverted operation	Weakens the claim instead of adding termination.

12. Answer C Concept 8

A team must check that input strings have balanced parentheses nested to any depth. A junior developer proposes a regular expression. ~~The input comes from a web form built last year.~~ Which statement is correct?

Target word: language class needed **Decides:** Unlimited nesting depth.

A	Absolute language	Regular languages have no memory for unlimited nesting.
B	Camouflage	The form's age does not change the language class.
C	Correct	Balanced nesting of any depth is not a regular language.
D	Adjacent category	More power than needed; context-free is enough.

Triage grid

Write each item number in the box it belongs to.

	Right	Wrong
Sure	Owled.	Concept inversion — top priority. A wrong mental model; it survives rewording.
Unsure / guess	Recognition, not mastery. Re-drill with the transfer.	Not learned yet.

Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

Trap	What it does	In this drill	Your misses
Dropped element	Leaves out a required term or step.	1	
Wrong stem number	A figure copied from the prompt that is not the answer.	0	
Adjacent category	The right idea in the wrong bucket.	7	
Inverted operation	Direction flipped: add/subtract, cause/effect, higher/lower.	6	
Absolute language	Always, never, guarantee — where judgment applies.	5	
Confusion set	Every option is real; the test is exact role-matching.	9	
Camouflage	Answers a stem fact that was never the question.	8	

Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

#	Concept	Rule	Trap	Tell
1	Compiled vs. interpreted	Compiled code is translated to machine code before running, favoring speed and standalone binaries; interpreted code is executed by an interpreter at run time, favoring flexibility.	Assuming lower-level (even assembly) is always the right business choice.	Requirements about speed, deployment form, or quick changes.
2	Functional vs. procedural	Functional style uses pure functions (same input, same output, no side effects) and immutable data; procedural style changes state step by step.	Calling any function that returns a value pure.	A function that also writes to globals, files, or clocks.
3	Static vs. dynamic typing	Static typing checks types before the program runs; dynamic typing finds type errors only when that code executes.	Confusing static/dynamic with strong/weak typing.	When the type error is caught.
4	Declarative query languages	A query language such as SQL states what result is wanted, and the engine decides how to get it.	Thinking the query writer controls the loop order.	A query describing the desired rows.
5	Object-oriented principles	Encapsulation hides state behind methods; inheritance shares structure; polymorphism lets one call run the right subtype's behavior.	Mixing up which principle a scenario shows.	One call on a base type producing different behavior per subtype.
6	Testing vs. formal verification	Testing can show bugs are present but not that they are absent; formal verification proves a program meets its specification for all inputs.	Treating many passing tests as proof, or treating verification as proof the spec is right.	A claim of correctness based on test counts.
7	Partial vs. total correctness	Partial correctness: if it terminates, the result is right; total correctness adds a proof that it always terminates.	Calling a proof total without a termination argument.	A proof that assumes the loop ends.
8	Regular vs. context-free languages	Regular expressions cannot match arbitrarily deep nesting; that needs a context-free grammar (a stack).	Believing a clever regex can handle any nesting depth.	Balanced or nested structure of unlimited depth.

Course scope (public description): Formal Languages Overview introduces students to programming language design and theory, focusing on formal semantics and type systems. It covers imperative, functional, and parallel languages, emphasizing techniques for proving language properties and verifying program specifications. Students will differentiate between functional and procedural languages, and explore compiled, interpreted, query, and assembly languages. The course also examines the structure and features of programming languages, including object-oriented programming principles. Learners will understand program correctness, testing, and verification, addressing type correctness. This course prepares students to assess programming languages against business requirements, enhancing software reliability and efficiency. Aimed at those interested in the theoretical underpinnings of programming languages, it equips students with knowledge to make informed decisions in software development and application.